

RestoMe

Application mobile React Native

Commande à table en temps réel

Dossier de remise — Projet React Native

Date limite : 28 mai 2026

1. Rappel du sujet

RestoMe est une application mobile React Native développée avec Expo. Elle permet aux clients d'un restaurant de scanner un QR code de table, rejoindre ou créer une session, consulter le menu, ajouter des plats avec des notes, envoyer une commande, suivre l'état des articles en temps réel et demander l'addition.

L'application contient aussi un mode cuisine caché, accessible uniquement via le QR code staff KITCHEN_001, où le personnel peut se connecter, voir les commandes en direct, changer le statut des articles, envoyer des messages aux clients, gérer le menu, les photos, les allergènes et la disponibilité des plats.

2. Fonctionnalités implémentées

Côté client

Fonctionnalité	Statut
Scan QR avec caméra	Réalisé avec expo-camera
Saisie manuelle du QR code	Réalisé
Détection QR cuisine caché	Réalisé
Création/reprise d'une session de table	Réalisé via Supabase
Menu par catégories	Réalisé : starter, main, dessert, drink
Détail d'un plat	Image, prix, allergènes, quantité, notes
Panier local	Réalisé avec React Context
Soumission de commande	Réalisé via Supabase
Historique local de commande	Réalisé avec SQLite
Suggestions basées sur l'historique	Réalisé
Allergènes sauvegardés localement	Réalisé
Mode clair / sombre	Réalisé avec SQLite
Suivi de commande en temps réel	Réalisé avec Supabase Realtime
Messages cuisine → client	Réalisé
Demande d'addition	Interface + alerte locale

Côté cuisine

Fonctionnalité	Statut
Connexion email / mot de passe	Réalisé avec Supabase Auth
Accès cuisine caché par QR code	Réalisé
Protection des routes cuisine	Réalisé dans le layout kitchen
Liste des tables actives	Réalisé
File de commandes en direct	Réalisé
Statistiques pending / preparing / ready	Réalisé
Changement statut article	Réalisé
Marquer un article indisponible	Réalisé
Envoyer un message au client	Réalisé
Fermer une session	Réalisé
Ajouter/modifier un article menu	Réalisé
Upload image article	expo-image-picker + Supabase Storage
Ajouter un allergène personnalisé	Réalisé
Rechercher dans le menu cuisine	Réalisé
Activer/désactiver disponibilité	Réalisé en temps réel

3. Justification technique

Le projet utilise React Native avec Expo pour créer une application mobile rapidement testable avec Expo Go. La navigation est gérée avec Expo Router, car le projet est organisé par fichiers dans le dossier app/, avec des routes client et un groupe séparé (kitchen) pour le mode cuisine.

Le projet utilise TypeScript, ce qui permet de définir clairement les types métier : MenuItem, Order, OrderItem, Session, Allergen, etc. Cela renforce la robustesse du code et facilite la maintenance.

Base de données distante

Le projet utilise Supabase avec @supabase/supabase-js. Il n'y a pas Axios dans le projet : tous les appels réseau passent par le client Supabase, par exemple .from('menu_items').select(...), .insert(...), .update(...). Ce choix évite une couche d'abstraction supplémentaire et bénéficie directement des fonctionnalités natives de Supabase (Realtime, Auth, Storage).

Authentification

L'authentification cuisine utilise Supabase Auth avec email/mot de passe. La session auth est persistée avec Expo SecureStore pour garantir un stockage sécurisé des tokens sur l'appareil.

Stockage local

Le stockage local utilise Expo SQLite. SQLite sert uniquement aux données locales du client : identifiant appareil, thème, historique de commandes et allergènes sauvegardés. Le menu et les commandes restent dans Supabase pour assurer la cohérence des données entre clients et cuisine.

État global

L'état global utilise React Context, organisé par responsabilité :

- **AuthContext** — état de connexion cuisine
- **SessionContext** — table/session client
- **CartContext** — panier local
- **ThemeContext** — thème clair/sombre
- **UserSettingsContext** — allergènes sauvegardés

Formulaires

Les formulaires cuisine utilisent React Hook Form et Zod pour valider les champs, notamment le login et le formulaire d'ajout/modification d'un article. Cette combinaison offre une validation typée, performante et déclarative.

Temps réel

Le temps réel utilise Supabase Realtime :

- **order_items** — mise à jour de la file cuisine
- **status_updates** — affichage des messages côté client
- **menu_items** — propagation des changements de disponibilité
- **session_joins** — channel broadcast pour notifier la cuisine quand une session est ouverte ou rejointe

4. Architecture

Structure principale

Dossier / Fichier	Rôle
app/	Routes Expo Router
app/(kitchen)/	Routes du mode cuisine
src/features/customer/	Écrans, composants et hooks client
src/features/kitchen/	Écrans, composants, hooks et utils cuisine
src/services/	Accès Supabase : menu, commandes, sessions, tables, auth
src/contexts/	États globaux React Context

Dossier / Fichier	Rôle
<code>src/lib/db.ts</code>	Initialisation et accès SQLite
<code>src/lib/supabase.ts</code>	Client Supabase
<code>src/types/index.ts</code>	Types TypeScript partagés

Flux client

- Le client ouvre l'app sur l'écran QR.
- Il scanne un QR code de table ou saisit le code manuellement.
- L'app cherche la table dans Supabase via `tables.qr_code`.
- Elle vérifie s'il existe une session ouverte ; si oui, le client peut la rejoindre, sinon une nouvelle session est créée.
- Le client consulte le menu, ajoute des articles au panier, puis valide.
- L'app crée ou retrouve la commande ouverte de la session.
- Les articles sont insérés dans `order_items` avec le statut `pending`.
- Le client suit l'état en direct sur l'écran `live order`.

Flux cuisine

- Le staff scanne le QR cuisine `KITCHEN_001`.
- L'app redirige vers `/login`.
- Le staff se connecte avec Supabase Auth.
- Le dashboard affiche les sessions ouvertes et commandes actives.
- Le staff change les statuts : `pending`, `preparing`, `ready`, `unavailable`.
- Le staff peut envoyer des messages au client via `status_updates`.
- Le staff peut fermer une session, ce qui ferme aussi la commande ouverte liée.

5. Auto-évaluation

Le tableau ci-dessous récapitule les fonctionnalités annoncées et leur état de réalisation effectif :

Fonctionnalité annoncée	Réalisée ?	Commentaire
Scan QR table	Oui	Caméra + saisie manuelle
Session par table	Oui	Supabase sessions
Menu par catégories	Oui	starter, main, dessert, drink
Détail plat avec allergènes	Oui	Image, prix, allergènes
Panier local	Oui	React Context

Fonctionnalité annoncée	Réalisée ?	Commentaire
Notes par article	Oui	Stockées dans order_items.notes
Soumission commande	Oui	Insertion Supabase
Commandes immuables côté client	Oui	Pas d'édition après envoi
Suivi temps réel client	Oui	order_items + status_updates
Suggestions personnalisées	Oui	Basées sur SQLite order_history
Thème clair/sombre	Oui	Sauvegardé en SQLite
Préférences allergènes locales	Oui	SQLite user_allergens
Connexion cuisine	Oui	Supabase Auth
Mode cuisine caché	Oui	QR staff uniquement
Dashboard cuisine live	Oui	Supabase Realtime
Changement statut article	Oui	Update order_items.status
Messages cuisine-client	Oui	Table status_updates
Gestion du menu	Oui	Création, édition, disponibilité
Upload photo menu	Oui	Supabase Storage
Ajout allergène	Oui	Table allergens
Demande d'addition	Partiel	Interface/alerte locale, pas persistée en base

6. Configuration à documenter

Le projet nécessite un fichier .env à la racine avec les variables suivantes :

EXPO_PUBLIC_SUPABASE_URL=https://cyixryvxdvttphkszmw.supabase.co

EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImN5aXh5eXZ4d2R2dHRwa2hzem13Iiwicm9sZSI6ImFub24iLCJpYXQiOiE3NzY5NDYyODcsImV4cCI6IjA5MjUxNzI4N30.If0y7WB0P6SvYc6IFyDcKpdhHbIPvfXT-dDVuZXfWr4

Commandes utiles

npm install

→ Installe les dépendances du projet.

npm run start

→ Lance le serveur de développement Expo.

npm run start:tunnel

→ Lance Expo en mode tunnel (pour tester sur un appareil physique avec Expo Go, notamment lorsque le réseau local n'est pas accessible).

npm run ts:check

→ Vérifie les types TypeScript de l'ensemble du projet.

npm test

→ Exécute la suite de tests.

Notes importantes sur le lancement

Remarque : il arrive que la commande **npm run start:tunnel** ne fonctionne pas correctement a cause des configurations ngrok, je n'ai pas pu trouver une solution pour celui-ci.

Solution recommandée : dans ce cas, il est conseillé d'utiliser **Android Studio** avec un émulateur Android (AVD). Après avoir lancé l'émulateur, exécuter **npm run start** puis appuyer sur la touche **a** dans le terminal Expo permet d'ouvrir directement l'application sur l'émulateur, sans dépendre du réseau local ni du tunnel.

Cette méthode est la plus fiable pour tester l'application lors de la soutenance et garantit un démarrage sans crash.

— Fin du dossier —